

TP d'optimisation avec NumPy

Mohamed Reda Lakouas

Octobre 2026

1 Introduction

Le but de ce TP est de programmer plusieurs méthodes d'optimisation avec NumPy. J'ai travaillé sur trois problèmes : la régression, la classification et le clustering. J'ai comparé GD, GD avec recherche du pas, SGD, Momentum, AdaGrad, RMSprop et Adam.

J'ai utilisé les graines 42, 123 et 2024 pour les petits jeux. Pour MNIST et Fashion-MNIST, j'ai d'abord fait un essai court avec 3000 images train, 1000 validation et 1000 test. Cet essai utilise une graine et deux époques. Il sert surtout à vérifier que tout le programme fonctionne.

Le code utilisé pour obtenir les résultats est disponible sur GitHub :

<https://github.com/Dev0am/optimisation-numerique-numpy>

2 Principe de l'optimisation

Le modèle possède des paramètres notés θ . La fonction $J(\theta)$ mesure l'erreur. Le gradient indique la direction où cette erreur augmente. Pour la faire diminuer, j'utilise la mise à jour suivante :

$$\theta_{\text{nouveau}} = \theta_{\text{ancien}} - \eta \nabla J(\theta).$$

Le nombre η est le pas d'apprentissage. S'il est trop petit, le programme avance lentement. S'il est trop grand, la perte peut devenir instable.

Dans le code, les sept méthodes utilisent la même boucle. Seule la façon de modifier le gradient change. Adam utilise par exemple une moyenne du gradient et une moyenne des gradients au carré.

J'ai aussi vérifié les gradients par différences finies avec $h = 10^{-5}$. Les erreurs obtenues sont entre 10^{-8} et 10^{-10} . Cela montre que les gradients programmés sont corrects.

3 Exercice 1 : régression

Pour OLS et Ridge, la perte utilisée est :

$$J(w) = \frac{\|Xw - y\|^2}{2n} + \frac{\lambda}{2} w^T P w.$$

Quand $\lambda = 0$, on obtient OLS. Pour Ridge, $\lambda > 0$. Le biais n'est pas pénalisé. J'ai aussi utilisé le noyau RBF pour représenter une relation non linéaire.

TABLE 1 – Comparaison des sept méthodes avec RBF sur Nonlinear.

Méthode	RMSE moyenne $\pm$ écart type	MAE	R^2	Temps (s)	Mises à jour	Gradients	Pertes
GD	0.683 ± 0.000	0.597	0.311	0.015	120	120	240
GD recherche	0.640 ± 0.000	0.560	0.396	0.015	120	120	240
SGD	0.662 ± 0.002	0.579	0.354	0.050	840	840	240
Momentum	0.575 ± 0.003	0.501	0.511	0.049	840	840	240
AdaGrad	0.592 ± 0.002	0.517	0.483	0.036	840	840	240
RMSprop	0.574 ± 0.009	0.502	0.514	0.039	840	840	240
Adam	0.565 ± 0.008	0.487	0.529	0.045	840	840	240

Le résultat principal est que RBF fonctionne mieux sur le jeu Nonlinear. C'est logique car OLS cherche seulement une relation linéaire. Sur Diabetes, Ridge et OLS donnent des résultats proches et sont meilleurs que le RBF essayé. Le meilleur résultat Diabetes est obtenu par Ridge avec Adam : RMSE = 53.740, MAE = 42.940 et $R^2 = 0.467$.

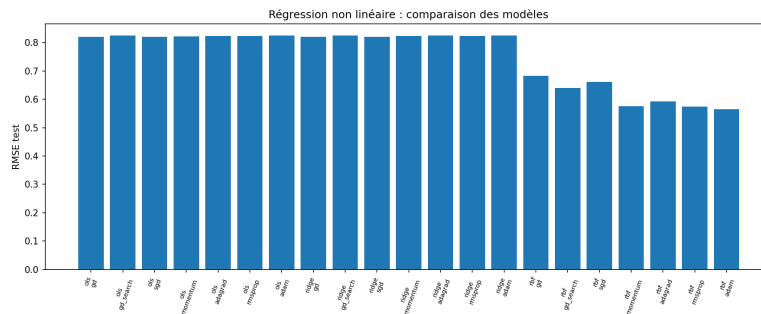


FIGURE 1 – Comparaison des modèles sur le jeu Nonlinear.

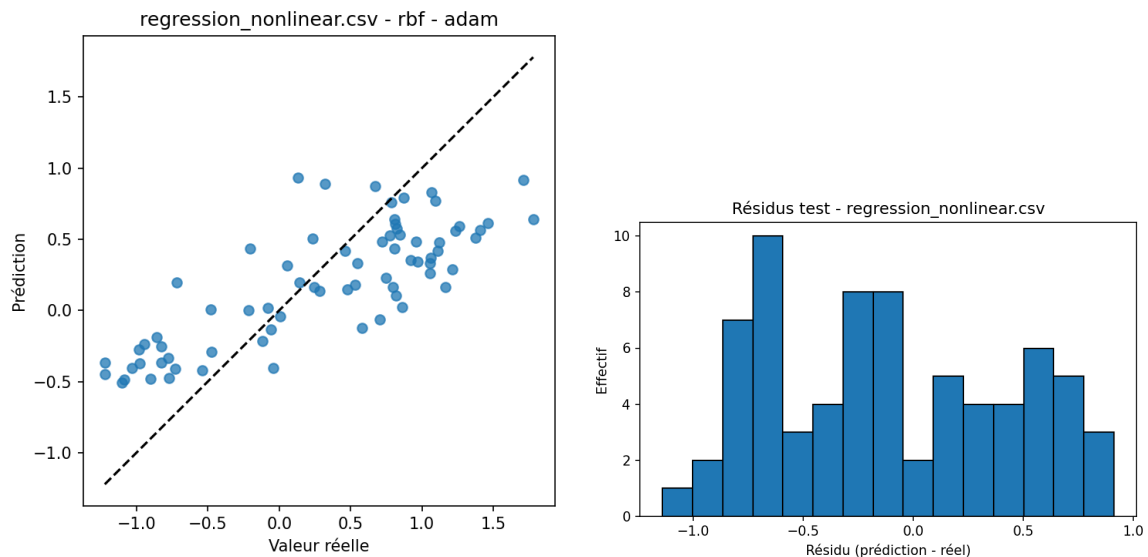


FIGURE 2 – Prédications et résidus du meilleur modèle.

4 Exercice 2 : classification

J'ai programmé un petit réseau avec deux couches cachées. Les architectures sont $2 \rightarrow 32 \rightarrow 16 \rightarrow 3$ pour Spiral, $4 \rightarrow 32 \rightarrow 16 \rightarrow 3$ pour Iris et $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ pour les images. Les couches cachées utilisent tanh. La dernière couche utilise softmax pour obtenir les probabilités des classes.

TABLE 2 – Comparaison des sept méthodes sur Spiral.

Méthode	Exactitude moyenne $\pm$ écart type	Précision macro	Rappel macro	F1 macro	Entropie	Temps (s)
GD	0.711 ± 0.000	0.711	0.711	0.711	0.571	0.118
GD recherche	0.989 ± 0.000	0.989	0.989	0.989	0.074	1.432
SGD	0.726 ± 0.005	0.726	0.726	0.726	0.525	0.117
Momentum	0.989 ± 0.000	0.989	0.989	0.989	0.083	0.226
AdaGrad	0.989 ± 0.000	0.989	0.989	0.989	0.045	0.129
RMSprop	0.989 ± 0.000	0.989	0.989	0.989	0.059	0.154
Adam	0.989 ± 0.000	0.989	0.989	0.989	0.085	0.192

Sur Spiral, les méthodes adaptatives apprennent mieux la forme des spirales que GD avec le pas choisi. Pour les images, RMSprop donne le meilleur résultat de l'essai court. Je ne considère pas ce résultat comme définitif car deux époques ne suffisent pas pour conclure. Sur Iris, AdaGrad donne 0.944 d'exactitude et de F1 macro. Dans l'essai images, RMSprop obtient 0.849 sur MNIST et 0.707 sur Fashion-MNIST.

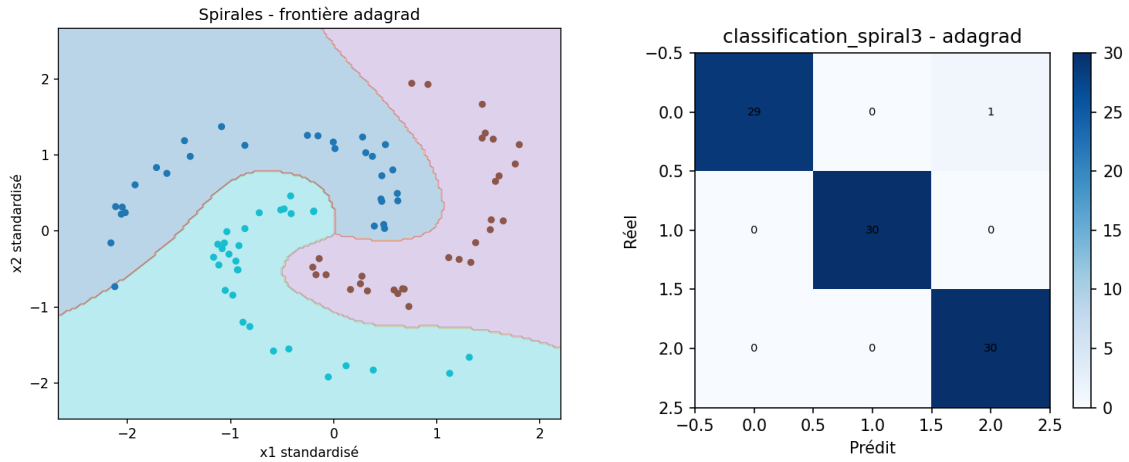


FIGURE 3 – Frontière de décision et matrice de confusion de Spiral.

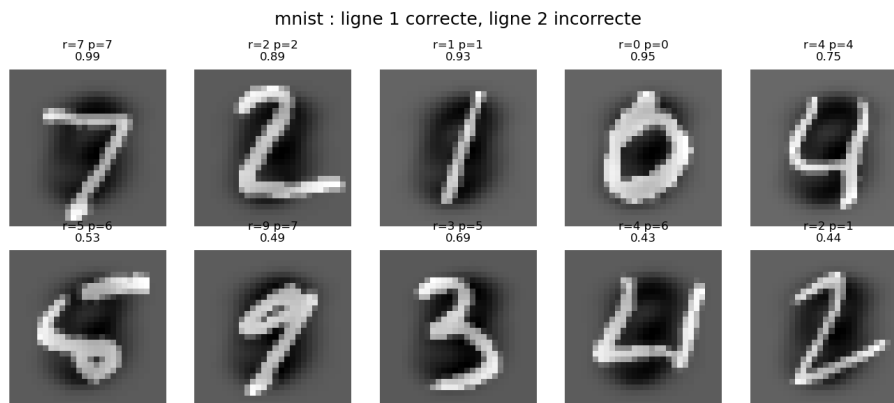


FIGURE 4 – Exemples MNIST bien et mal classés.

5 Exercice 3 : clustering

Pour éviter un grand tableau en mémoire, les distances sont calculées avec :

$$D_{ik} = \|x_i\|^2 + \|c_k\|^2 - 2x_i^T c_k.$$

Les points sont associés de façon douce aux centroïdes pendant l'apprentissage. À la fin, chaque point reçoit le centroïde le plus proche. Les vraies classes ne sont utilisées qu'après l'apprentissage pour calculer ARI et NMI. J'ai aussi programmé Lloyd comme méthode de comparaison.

TABLE 3 – Comparaison sur Blobs. Lloyd est ajouté comme référence.

Méthode	Perte	Inertie	Silhouette	ARI $\pm$ écart type	NMI	Temps (s)
GD	0.656	0.383	0.676	0.970 ± 0.014	0.954	0.011
GD recherche	0.564	0.290	0.679	0.975 ± 0.000	0.960	0.095
SGD	0.565	0.291	0.679	0.975 ± 0.000	0.960	0.023
Momentum	0.564	0.290	0.679	0.975 ± 0.000	0.960	0.023
AdaGrad	0.565	0.291	0.679	0.975 ± 0.000	0.960	0.024
RMSprop	0.564	0.290	0.679	0.975 ± 0.000	0.960	0.024
Adam	0.564	0.290	0.679	0.975 ± 0.000	0.960	0.025
Lloyd	0.564	0.290	0.679	0.975 ± 0.000	0.960	0.000

Blobs est le jeu le plus facile car ses trois groupes sont bien séparés. Wine est plus sensible à l’initialisation. Sur les images, Lloyd est meilleur dans ce petit essai. Une petite inertie ne garantit pas forcément une bonne valeur d’ARI, car les deux mesures ne représentent pas exactement la même chose. Sur Wine, le meilleur ARI moyen est 0.897 avec Momentum. Dans les essais images, Lloyd obtient un ARI de 0.389 sur MNIST et 0.326 sur Fashion-MNIST.

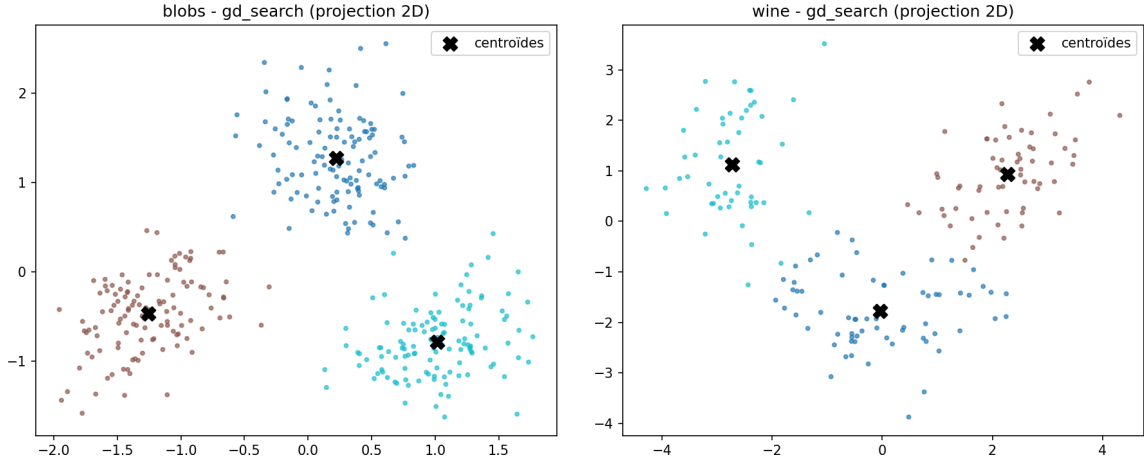


FIGURE 5 – Clustering de Blobs et projection PCA de Wine.

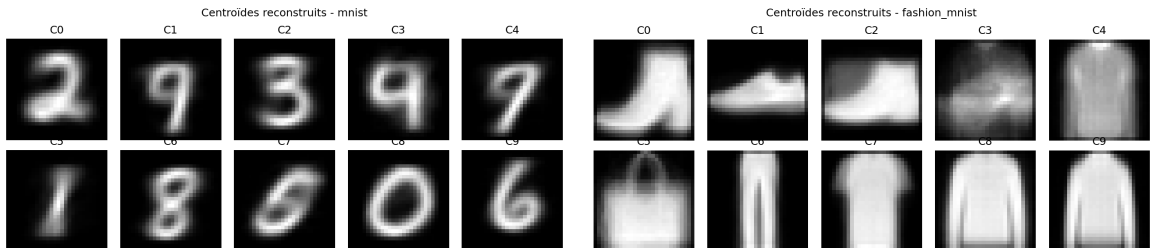


FIGURE 6 – Centroïdes obtenus sur les deux jeux d’images.

6 Conclusion

Ce TP m’a permis de comprendre que le choix du modèle est très important. Par exemple, changer OLS par RBF améliore beaucoup le jeu Nonlinear. J’ai aussi vu que le nombre d’époques ne suffit pas pour comparer les méthodes : une époque de GD contient une seule mise à jour, alors que SGD contient plusieurs mini-lots.

Les méthodes adaptatives donnent souvent de bons résultats rapidement, mais elles dépendent du pas choisi. Le clustering dépend aussi de l'initialisation. Les moyennes et écarts types sont calculés sur les graines 42, 123 et 2024 pour les méthodes stochastiques. Pour les méthodes déterministes, une seule exécution est suffisante avec la même initialisation. J'ai gardé séparément le nombre de mises à jour, les appels au gradient et les appels à la perte, car une époque de GD n'a pas le même coût qu'une époque de SGD. La principale limite de mes résultats est le budget court utilisé pour MNIST et Fashion-MNIST. Le code contient un mode `full` pour lancer l'expérience avec les partitions complètes et trois graines.

Commande d'exécution

```
python3 run_all.py --profile quick
python3 run_all.py --profile full
```

Dépôt du code : <https://github.com/Dev0am/optimisation-numerique-numpy>